

Task 1: Install TensorFlow & Keras and Create a 2-Layer Neural Network

Installation

Open your terminal or command prompt and run:

```
pip install tensorflow
```

Verify the installation:

```
import tensorflow as tf
```

```
print(tf.__version__)
```

Create a Simple Neural Network Using Keras Sequential API

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
# Create Model
```

```
model = Sequential([
```

```
    Dense(4, activation='relu', input_shape=(3,)), # Hidden Layer
```

```
    Dense(1, activation='sigmoid')                 # Output Layer
```

```
])
```

```
# Compile Model
```

```
model.compile(
```

```
    optimizer='adam',
```

```
    loss='binary_crossentropy',
```

```
    metrics=['accuracy']
```

```
)
```

```
# Display Model Summary
```

```
model.summary()
```

Architecture

Input Layer (3 Features)



Hidden Layer (4 Neurons, ReLU)



Output Layer (1 Neuron, Sigmoid)

Task 2: Print Weights Before and After One Training Step

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# Build Model
```

```
model = Sequential([
    Dense(3, activation='relu', input_shape=(2,)),
    Dense(1, activation='sigmoid')
])
```

```
model.compile(
    optimizer='sgd',
    loss='binary_crossentropy'
)
```

```
# Dummy Data
```

```
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])
```

```
y = np.array([0,1,1,0])
```

```
print("Weights BEFORE Training\n")
```

```
for layer in model.layers:
    print(layer.get_weights())
```

```
# Train One Epoch
```

```
model.fit(X, y, epochs=1, verbose=0)
```

```
print("\nWeights AFTER Training\n")
```

```
for layer in model.layers:
    print(layer.get_weights())
```

Observation

- Before training → Weights are randomly initialized.
- After one training step → Weights change slightly based on the gradient descent update rule.

Task 3: Manual Gradient Calculation Using Chain Rule

Given

Input (x) = 2

Weight (w) = 0.5

Bias (b) = 1

Target (y) = 1

Activation = Sigmoid

Step 1: Linear Output

$$z = wx + b$$
$$z = (2)(0.5) + 1 = 2$$

Step 2: Sigmoid Output

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$
$$a = \frac{1}{1 + e^{-2}}$$
$$a \approx 0.8808$$

Step 3: Loss

Use Mean Squared Error

$$L = \frac{1}{2}(a - y)^2$$
$$L = \frac{1}{2}(0.8808 - 1)^2$$

Step 4: Chain Rule

$$\frac{dL}{dw} = \frac{dL}{da} \cdot \frac{da}{dz} \cdot \frac{dz}{dw}$$

Now compute each term.

(i)

$$\frac{dL}{da} = a - y = 0.8808 - 1 = -0.1192$$

(ii)

Sigmoid derivative

$$\begin{aligned} \frac{da}{dz} &= a(1 - a) \\ &= 0.8808(1 - 0.8808) = 0.1050 \end{aligned}$$

(iii)

$$\frac{dz}{dw} = x = 2$$

Final Gradient

$$\begin{aligned} \frac{dL}{dw} &= (-0.1192)(0.1050)(2) \\ &= -0.025 \end{aligned}$$

Final Answer

Gradient $dL/dw \approx -0.025$

A negative gradient indicates that increasing the weight would reduce the loss, so gradient descent will increase the weight during the update.

Task 4: Simulate Vanishing Gradient

```
import numpy as np
```

```
def sigmoid(x):
```

```

    return 1/(1+np.exp(-x))

def simulate_vanishing_gradient(x=0.9, layers=10):

    print("Layer Outputs\n")

    for i in range(layers):
        x = sigmoid(x)
        print(f"Layer {i+1}: {x:.6f}")

    return x

final_output = simulate_vanishing_gradient()

print("\nFinal Output:", final_output)

```

Sample Output

```

Layer 1 : 0.710949
Layer 2 : 0.670611
Layer 3 : 0.661640
Layer 4 : 0.659628
Layer 5 : 0.659176
...
Layer10 : 0.659046

```

Observation

Repeated sigmoid activations compress values into a narrow range, and the sigmoid derivative becomes small. During backpropagation, multiplying many small derivatives causes gradients to shrink, which is known as the **vanishing gradient problem**.

Task 5: Exploding Gradient Demonstration

```

import numpy as np

# Large initial weights
weights = np.full((5,), 10.0)

gradient = 1.0

print("Initial Gradient:", gradient)

for layer in range(10):

```

```
gradient *= weights[0]  
print(f"Layer {layer+1}: Gradient = {gradient:.2e}")
```

Sample Output

```
Initial Gradient: 1.0  
Layer 1: Gradient = 1.00e+01  
Layer 2: Gradient = 1.00e+02  
Layer 3: Gradient = 1.00e+03  
Layer 4: Gradient = 1.00e+04  
...  
Layer10: Gradient = 1.00e+10
```